VITyarthi | Open Source Software

The Open Source Audit

A Capstone Project for OSS NGMC Course

Course: Open Source Software | Unit Coverage: 1 – 5

Max Marks: 100

Student Name	**Aditya Singh**
Registration Number	**24BCE11444**
Slot	A13
Date	**29/03/2026**

1. Introduction

When I started learning programming and working on small projects, I didn't really think much about how developers manage large codebases or work together on the same project. At that time, I used to just save different versions of my code manually, like `final.cpp`, `final2.cpp`, and sometimes even `final_final.cpp`. It was messy and confusing. That's when I first came across **Git**, and I realized that there is actually a proper system designed to solve this problem.

As I started reading more about Git, I understood that it is not just a tool, but something that has completely changed the way developers work. It helps in tracking changes in code, managing different versions, and most importantly, allowing multiple people to work on the same project without creating chaos. What I found interesting is that something which looks so simple from outside is actually solving a very complex problem behind the scenes.

The more I explored Git, the more I became curious about its background. I wanted to know why it was created, what problems existed before it, and how it became so popular. I also found out that Git is open-source, which means anyone can study its code, modify it, and use it freely. This made me realize how important open-source software is in the development world, because most of the tools we use today are built on this idea of sharing and collaboration.

That is the main reason I chose Git for this project. Instead of just learning how to use commands, I wanted to understand the story behind it — how it started, what challenges it solved, and how it became such an important part of software development. In this report, I will be analyzing Git from different perspectives, including its origin, its license, its role in Linux systems, and its overall impact in the open-source ecosystem.

Along with the theory, I will also be working on some practical tasks using Linux and shell scripting. This will help me connect the concepts with real-world usage and understand how these tools actually work at the system level. Overall, this project is not just about completing an assignment, but about understanding something that I will be using throughout my career as a developer.

2. Software :

Your choice	Write your chosen software here before submission: _____Git_____

3. Project Report

Open-source software forms the backbone of modern computing, powering everything from operating systems to web applications and development tools. Unlike proprietary software, open-source software is built collaboratively and shared freely, allowing anyone to study, modify, and distribute the code.

One of the most important open-source tools used in software development today is **Git**. Git is a distributed version control system that helps developers track changes in their code, collaborate with others, and maintain different versions of a project efficiently.

This report presents a structured audit of Git, exploring its origin, philosophy, licensing model, and its role in the open-source ecosystem. Along with theoretical analysis, the report also includes practical insights gained by using Git on a Linux system and implementing shell scripts to demonstrate key concepts of open-source tools.

The objective of this project is not only to understand how Git works but also to explore the broader ideas of openness, collaboration, and freedom that define the open-source movement.

Part A — Origin and Philosophy (Units 1 & 2)

A1. The problem this software was created to solve

While learning more about Git, I became curious about why it was even created in the first place. At first, I thought it was just another tool for managing code, but after reading about its background, I realized that it was actually built to solve some serious problems that developers were facing at that time.

Before Git existed, developers were using version control systems like CVS and Subversion. These systems worked, but they had a major limitation — they were centralized. This means all the project data was stored on a single server, and every developer had to connect to that server

to make changes. If the server went down, the whole work would stop. Also, working without an internet connection was very difficult, which made development less flexible.

Another issue was speed. These older systems were quite slow, especially when dealing with large projects. Tasks like committing changes, creating branches, or merging code used to take a lot of time. For small projects, this might not be a big issue, but for something as large as the Linux kernel, it became a serious problem.

One more important issue was related to licensing. The Linux development team was using a tool called BitKeeper, which was not completely open-source. Due to some licensing disagreements, they lost access to it in 2005. This created a big problem because they suddenly didn't have a proper tool to manage their code.

This situation led to the creation of Git by Linus Torvalds. Instead of trying to fix the existing systems, he decided to build a completely new one from scratch. His goal was to create something that was fast, reliable, and most importantly, free and open-source.
Git introduced a completely different approach by using a distributed system. In this model, every developer has a full copy of the entire project, including its history. This means there is no dependency on a single server, and developers can work offline without any problem. This also makes the system more secure and reliable, because even if one copy is lost, others still exist.

Another thing that makes Git powerful is its speed. Most operations are performed locally, which makes them much faster compared to older systems. It also uses a hashing mechanism to keep track of changes, which ensures that the data remains safe and unchanged.

After understanding all this, I realized that Git was not just created randomly. It was built as a solution to real problems faced by developers, especially in large open-source projects. It solved issues related to speed, flexibility, and control, and because of that, it quickly became the most widely used version control system in the world.

In my opinion, the creation of Git shows how open-source problems can lead to powerful innovations. Instead of depending on proprietary tools, developers created something better and shared it with everyone, which is why Git is so widely used today.

A2. The license — what it actually says

After understanding why Git was created, I wanted to know under what kind of license it is released, because in open-source software, the license actually decides what we can and cannot do with the software. While reading about it, I found that Git is released under the GNU General Public License v2, which is commonly known as GPL v2.

At first, I thought "free software" simply means that we don't have to pay money for it, but after reading more carefully, I realized that it is more about freedom than price. The GPL license gives users certain rights, which are known as the four freedoms of free software. These include the freedom to use the software for any purpose, study how it works, modify it according to our

needs, and share it with others. In the case of Git, all these freedoms are clearly allowed, which makes it a true open-source software.

One thing that I found interesting about the GPL license is that it follows something called a “copyleft” rule. In simple words, if someone modifies Git and distributes it, they are required to share their modified version under the same license. This means they cannot take Git, improve it, and then make it closed-source. Personally, I feel this is a very smart rule because it ensures that the software always remains open and benefits everyone, not just a single company.

Another question that came to my mind was whether companies can use Git for business purposes. After reading about it, I understood that yes, companies are allowed to use Git and even build products around it. However, they cannot hide the source code if they are distributing a modified version. This creates a balance where businesses can grow, but user freedom is still protected.

While comparing licenses, I also came across the MIT License, which is another popular

open-source license. The main difference I noticed is that MIT is more permissive. It allows people to modify the software and even make it proprietary if they want. On the other hand, GPL is stricter because it forces all modified versions to remain open-source. If I had to choose, I think I would prefer GPL for a big project because it protects openness, but for smaller projects, MIT might be easier to use.

There is also a concept that I found quite interesting — “free as in free beer” and “free as in freedom.” Earlier, I used to think free just means no cost, but now I understand that real value comes from freedom. Git is a good example of “free as in freedom” because it gives complete control to users, not just free access.

In conclusion, the license of Git plays a very important role in its success. The GPL v2 license ensures that Git remains open, transparent, and accessible to everyone. It protects the rights of users and encourages developers to contribute without fear. After learning about this, I feel that the license is not just a legal document, but a key part of the philosophy behind open-source software.

Features	GPL License	MIT License
Open-source required	Yes	No
Freedom level	High	Moderate

A3. The ethics of open source

After learning about the origin and license of Git, I started thinking about the bigger picture of open-source software. It is not just about code or technology, but also about ideas, values, and ethics. Open source represents a way of thinking where knowledge is shared freely, and people collaborate instead of competing in isolation.

One question that came to my mind was whether all software should be open-source. On one side, I feel that open-source software is very beneficial because it promotes transparency and trust. Anyone can check the code, find bugs, and improve it. This makes the software more secure and reliable. It also helps students and beginners learn from real-world code, which is something I personally find very helpful.

However, on the other side, I also understand that not all software can be open-source. Many companies invest a lot of time and money into building their products, and they need to earn profit to survive. If everything becomes open-source, it might be difficult for some businesses to maintain their competitive advantage. So, I think there should be a balance between open-source and proprietary software.

Another interesting point is whether it is ethical for companies to make money using open-source software. Many big companies use tools like Git and build their products on top of them. In my opinion, this is not completely wrong, because these companies often contribute back to the community by improving the software or supporting its development. At the same time, it can feel unfair if a company benefits a lot from community work but does not contribute anything in return. So, I believe companies should give back to the open-source community in some form.

There is also a concept called “standing on the shoulders of giants,” which I found very meaningful in this context. It means building new ideas on top of existing work. Open-source software makes this possible because developers do not have to start from zero. They can use existing tools like Git and focus on creating something new. In my opinion, this does not reduce innovation; instead, it actually increases it, because people can move faster and build better solutions.

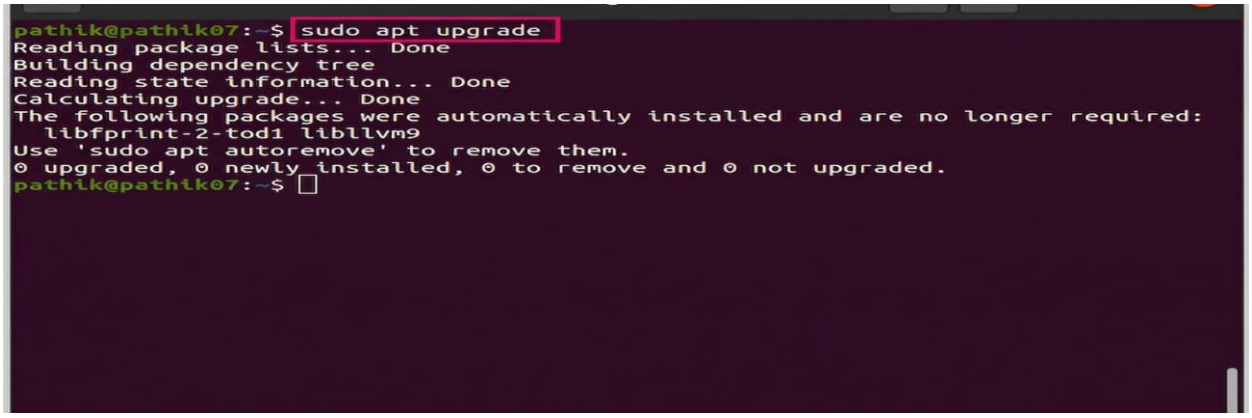
After thinking about all these points, I feel that open-source software like Git represents a very positive approach to development. It encourages learning, sharing, and collaboration. At the same time, it also shows that technology is not just about writing code, but also about making decisions that affect the entire community.

In conclusion, the ethics of open source are not black and white. There are both advantages and challenges, but overall, I believe that open-source software plays a very important role in the growth of technology. It creates opportunities for developers like me to learn, contribute, and become part of a global community.

Part B — Linux Footprint (Unit 2)

After understanding the theory behind Git, I wanted to see how it actually works on a Linux system. Since Git is mainly used in development environments, it is very closely connected with Linux. So, I installed Git on my system and explored how it is stored, how it runs, and how it is managed.

To install Git, I used the package manager available in my Linux system. In my case, I used the `apt` command:



The Open Source Audit | VITyarthiPage 7

The installation process was simple and completed within a few seconds. After installing, I verified it using the command `git --version`, which confirmed that Git was successfully installed on my system.

When I explored where Git is located in the system, I found that its main executable file is stored in the `/usr/bin/git` directory. This is the standard location where most system binaries are stored in Linux. Apart from this, Git also has configuration files. One of the important configuration files is `~/.gitconfig`, which stores user-specific settings like username and email. There is also a system-wide configuration file located at `/etc/gitconfig`.

Unlike some other software, Git does not run as a background service. It is a command-line tool that runs only when we execute a command. For example, when I run commands like `git init`, `git add`, or `git commit`, Git performs the required operation and then stops. This makes it lightweight and efficient, as it does not consume system resources when not in use.

Another important thing I noticed is that Git runs under the current logged-in user. This means all operations performed by Git are based on the permissions of that user. This is important for security because it ensures that only authorized users can make changes to a repository.

Regarding updates, Git is maintained by an active open-source community. Whenever there are security updates or new features, they are released through the package manager. By running commands like `sudo apt update` and `sudo apt upgrade`, I can easily keep Git updated on my system.

From my observation, Git fits very naturally into the Linux environment. It follows the standard Linux directory structure, uses existing system tools, and works smoothly with the command line. This shows how open-source tools are designed to integrate well with each other.

Overall, this practical exploration helped me understand that Git is not just a theoretical tool, but something that is deeply connected with the Linux system. It is simple to install, easy to use, and efficiently managed, which makes it a powerful tool for developers.

Part C — The FOSS Ecosystem (Units 3 & 4)

After exploring how Git works on a Linux system, I wanted to understand where it stands in the bigger picture of open-source software. One thing I realized is that Git does not work alone. It is part of a much larger ecosystem where different tools and technologies are connected and depend on each other.

Git itself depends on some basic system components like the C programming language libraries, file system, and shell environment. Since it is written in C, it uses standard system libraries to perform operations efficiently. It also relies on the underlying Linux file system to store repositories and track changes. These dependencies are not very complex, but they are essential for Git to function properly.

What I found more interesting is what Git has enabled over time. Many popular platforms and tools are built around Git. For example, platforms like GitHub and GitLab use Git as their core system to manage repositories and collaboration. These platforms have made it much easier for developers to share code, contribute to projects, and work together from different parts of the world.

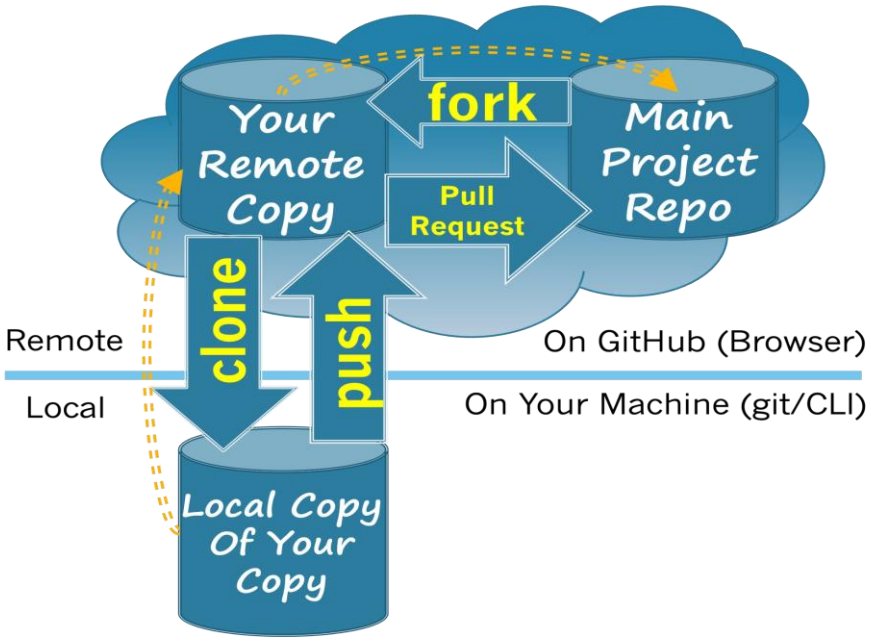
Git also plays a very important role in modern software development. Almost every development project, whether it is web development, mobile apps, or system software, uses Git for version control. It helps developers manage code changes, fix bugs, and experiment with new features without affecting the main project.

In terms of the web ecosystem, Git indirectly supports many technologies. For example, when developers work on websites or applications using the LAMP stack (Linux, Apache, MySQL, PHP), Git is often used to manage the source code of these projects. It helps in maintaining different versions and collaborating with team members.

Another important aspect of Git is its community. Git has a large and active open-source community that continuously works on improving it. Developers contribute by fixing bugs, adding new features, and maintaining the system. Discussions and updates often happen through mailing lists and online forums. Even though Git was originally created by Linus Torvalds, today it is maintained by a global community.

From my understanding, Git is not just a tool but a foundation for many other tools and platforms. It has created an environment where developers can collaborate easily and build new technologies on top of it.

In conclusion, the FOSS ecosystem around Git shows how powerful open-source collaboration can be. Git connects with many tools, supports modern development practices, and is maintained by a strong community. This makes it one of the most important components in today's software development world.



Part D — Open Source vs Proprietary (Critical Analysis)

Dimension	Open Source	Proprietary Alternative
Cost	*Completely free to use, modify, and distribute*	*Usually paid or requires license subscription*
Security (who can audit the code?)	*Source code is open, so anyone can inspect and find vulnerabilities*	*Source code is hidden, only company can audit*
Support and reliability	*Community support (forums, documentation, contributors)*	*Official customer support provided by company*
Freedom to modify	*Full freedom to modify and customize the software*	*No or very limited modification allowed*
Community vs corporate control	*Controlled and improved by global developer community*	*Controlled by a single company or organization*
Your overall verdict	*Flexible, transparent, and developer-friendly*	*More controlled but less flexible*

After comparing Git with proprietary version control tools, I feel that Git clearly has more advantages, especially for developers and students. The biggest benefit of Git is the freedom it provides. Since it is open-source, anyone can use it, modify it, and even study how it works internally. This makes it very useful not only for development but also for learning purposes.

On the other hand, proprietary tools may provide better customer support and sometimes easier setup, but they come with restrictions. Users are dependent on the company, and they do not have control over the software. In long-term use, this can become a limitation, especially in large or collaborative projects.

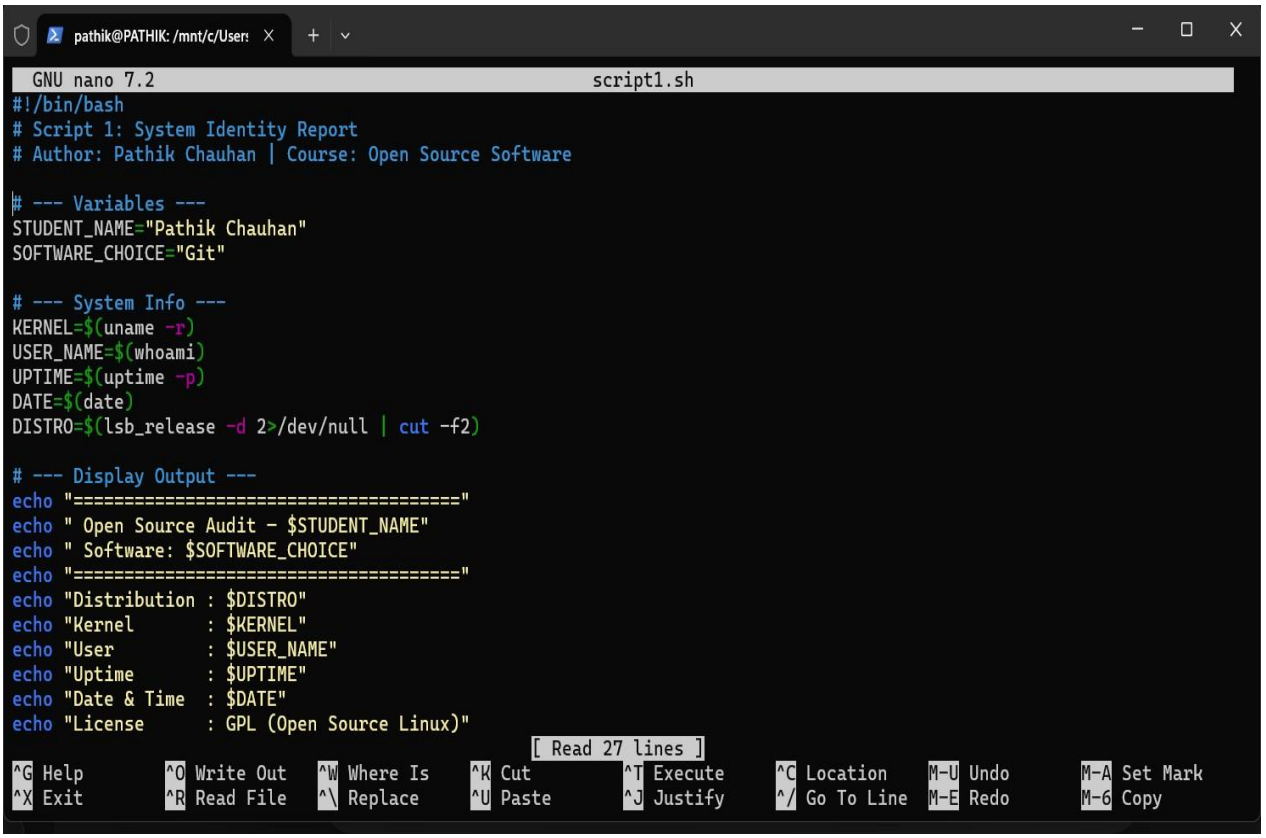
In my opinion, Git is a better choice for real-world development because it is flexible, reliable, and widely supported by the developer community. I would definitely recommend using Git for both personal and professional projects. Also, after learning about its open-source nature, I would be interested in contributing to such projects in the future, even if it is in a small way.

OUTPUT :

4. Shell Script Tasks

Script 1 — System Identity Report (Unit 1 & 2)

Starter structure



This script displays basic system information such as kernel version, user name, uptime, and Linux distribution. It uses variables, command substitution, and echo statements for formatted output.

```
pathik@PATHIK: /mnt/c/Users
Building dependency tree... Done
Reading state information... Done
63 packages can be upgraded. Run 'apt list --upgradable' to see them.
pathik@PATHIK:/mnt/c/Users/pathi$ sudo apt install git -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
git is already the newest version (1:2.43.0-1ubuntu7.3).
git set to manually installed.
0 upgraded, 0 newly installed, 0 to remove and 63 not upgraded.
pathik@PATHIK:/mnt/c/Users/pathi$ git --version
git version 2.43.0
pathik@PATHIK:/mnt/c/Users/pathi$ mkdir oss-project
pathik@PATHIK:/mnt/c/Users/pathi$ cd oss-project
pathik@PATHIK:/mnt/c/Users/pathi/oss-project$ nano script1.sh
pathik@PATHIK:/mnt/c/Users/pathi/oss-project$ chmod +x script1.sh
chmod: changing permissions of 'script1.sh': Operation not permitted
pathik@PATHIK:/mnt/c/Users/pathi/oss-project$ ./script1.sh
=====================================
 Open Source Audit — Pathik Chauhan
 Software: Git
=====================================
Distribution : Ubuntu 24.04.4 LTS
Kernel       : 6.6.87.2-microsoft-standard-WSL2
User         : pathik
Uptime       : up 15 minutes
Date & Time  : Sat Mar 28 19:07:45 UTC 2026
License      : GPL (Open Source Linux)
=====================================
pathik@PATHIK:/mnt/c/Users/pathi/oss-project$ |
```

OUTPUT :

Script 2 — FOSS Package Inspector (Unit 2)

Starter structure

```
pathik@PATHIK: /mnt/c/Users
  GNU nano 7.2                                    script2.sh
#!/bin/bash
# Script 2: FOSS Package Inspector

PACKAGE="git"

# Check if installed
if dpkg -l | grep -q "^ii  $PACKAGE"; then
    echo "$PACKAGE is installed."
    dpkg -l | grep "$PACKAGE"
else
    echo "$PACKAGE is NOT installed."
fi

# Case statement
case $PACKAGE in
    git)
        echo "Git: Distributed version control system used worldwide"
        ;;
    apache2)
        echo "Apache: Web server powering the internet"
        ;;
    mysql-server)
        echo "MySQL: Open-source database system"
        ;;
    *)
        echo "Unknown package"
        ;;
esac

^G Help      ^O Write Out   ^W Where Is    ^K Cut       ^T Execute   ^C Location   M-U Undo     M-A Set Mark
^X Exit      ^R Read File   ^\ Replace     ^U Paste     ^J Justify   ^/ Go To Line M-E Redo     M-6 Copy
```

This script checks whether Git is installed on the system using conditional statements. It also uses a case statement to display information about the package.

OUTPUT :

```
========================================
pathik@PATHIK:/mnt/c/Users/pathi/oss-project$ nano script2.sh
pathik@PATHIK:/mnt/c/Users/pathi/oss-project$ chmod +x script2.sh
chmod: changing permissions of 'script2.sh': Operation not permitted
pathik@PATHIK:/mnt/c/Users/pathi/oss-project$ ./script2.sh
git is installed.
ii  fonts-ubuntu                  0.869+git20240321-0ubuntu1              all          sans-serif font set from Ubuntu
ii  git                           1:2.43.0-1ubuntu7.3                     amd64        fast, scalable, distributed revis
ion control system
ii  git-man                       1:2.43.0-1ubuntu7.3                     all          fast, scalable, distributed revis
ion control system (manual pages)
ii  librtmp1:amd64                2.4+20151223.gitfa8646d.1-2build7       amd64        toolkit for RTMP streams (shared
library)
ii  libtiff6:amd64                4.5.1+git230720-4ubuntu2.4              amd64        Tag Image File Format (TIFF) libr
ary
ii  libwebp7:amd64                1.3.2-0.4build3                         amd64        Lossy compression of digital phot
ographic images
ii  lshw                          02.19.git.2021.06.19.996aaad9c7-2build3 amd64        information about hardware config
uration
Git: Distributed version control system used worldwide
pathik@PATHIK:/mnt/c/Users/pathi/oss-project$
```

Script 3 — Disk and Permission Auditor (Unit 2)

Starter structure

```
pathik@PATHIK: /mnt/c/Users

  GNU nano 7.2                                    script3.sh
#!/bin/bash
# Script 3: Disk and Permission Auditor

DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")

echo "Directory Audit Report"
echo "----------------------"

for DIR in "${DIRS[@]}"; do
    if [ -d "$DIR" ]; then
        PERMS=$(ls -ld $DIR | awk '{print $1, $3, $4}')
        SIZE=$(du -sh $DIR 2>/dev/null | cut -f1)
        echo "$DIR => Permissions: $PERMS | Size: $SIZE"
    else
        echo "$DIR does not exist"
    fi
done

# Check Git binary
if [ -f "/usr/bin/git" ]; then
    echo ""
    echo "Git binary exists:"
    ls -l /usr/bin/git
else
    echo "Git not found in /usr/bin"
fi

^G Help      ^O Write Out   ^W Where Is   ^K Cut       ^T Execute    ^C Location   M-U Undo      M-A Set Mark
^X Exit      ^R Read File   ^\ Replace    ^U Paste     ^J Justify    ^/ Go To Line M-E Redo      M-6 Copy
```

This script uses a for loop to analyze system directories and display their size and permissions. It also checks whether Git is present in the system.

OUTPUT :

```
pathik@PATHIK:/mnt/c/Users/pathi/oss-project$ nano script3.sh
pathik@PATHIK:/mnt/c/Users/pathi/oss-project$ chmod +x script3.sh
chmod: changing permissions of 'script3.sh': Operation not permitted
pathik@PATHIK:/mnt/c/Users/pathi/oss-project$ ./script3.sh
Directory Audit Report
----------------------
/etc => Permissions: drwxr-xr-x root root | Size: 4.3M
/var/log => Permissions: drwxrwxr-x root syslog | Size: 17M
/home => Permissions: drwxr-xr-x root root | Size: 36K
/usr/bin => Permissions: drwxr-xr-x root root | Size: 95M
/tmp => Permissions: drwxrwxrwt root root | Size: 24K

Git binary exists:
-rwxr-xr-x 1 root root 4066232 Jul  2  2025 /usr/bin/git
pathik@PATHIK:/mnt/c/Users/pathi/oss-project$
```

Script 4 — Log File Analyzer (Units 2 & 5)

Starter structure

```
pathik@PATHIK: /mnt/c/Users
  GNU nano 7.2                                    script4.sh
#!/bin/bash
# Script 4: Log File Analyzer

LOGFILE=$1
KEYWORD=${2:-"error"}
COUNT=0

if [ ! -f "$LOGFILE" ]; then
    echo "Error: File not found."
    exit 1
fi

while IFS= read -r LINE; do
    if echo "$LINE" | grep -iq "$KEYWORD"; then
        COUNT=$((COUNT + 1))
    fi
done < "$LOGFILE"

echo "Keyword '$KEYWORD' found $COUNT times."

echo ""
echo "Last 5 matching lines:"
grep -i "$KEYWORD" "$LOGFILE" | tail -5

                                  [ Read 23 lines ]
^G Help      ^O Write Out   ^W Where Is   ^K Cut       ^T Execute    ^C Location   M-U Undo      M-A Set Mark
^X Exit      ^R Read File   ^\ Replace    ^U Paste     ^J Justify    ^/ Go To Line M-E Redo      M-6 Copy
```

This script reads a log file line by line and counts occurrences of a keyword using a loop and conditional statements. It also displays the last matching lines.

OUTPUT :

```
pathik@PATHIK: /mnt/c/Users

pathik@PATHIK:/mnt/c/Users/pathi/oss-project$ nano script4.sh
pathik@PATHIK:/mnt/c/Users/pathi/oss-project$ echo "System started" > log.txt
echo "Error: file missing" >> log.txt
echo "User login success" >> log.txt
echo "error: connection failed" >> log.txt
echo "Warning: low memory" >> log.txt
echo "Error: disk issue" >> log.txt
pathik@PATHIK:/mnt/c/Users/pathi/oss-project$ ./script4.sh log.txt
Keyword 'error' found 3 times.

Last 5 matching lines:
Error: file missing
error: connection failed
Error: disk issue
pathik@PATHIK:/mnt/c/Users/pathi/oss-project$
```

Script 5 — The Open Source Manifesto Generator (Unit 5)

Starter structure

```
pathik@PATHIK: /mnt/c/Users

  GNU nano 7.2                                                    script5.sh
#!/bin/bash
# Script 5: Open Source Manifesto Generator
# Author: Pathik Chauhan

echo "===== Open Source Manifesto Generator ====="
echo ""

# Take user input
read -p "1. Which tool do you use? " TOOL
read -p "2. What does freedom mean to you? " FREEDOM
read -p "3. What do you want to build? " BUILD

DATE=$(date '+%d %B %Y')
USER=$(whoami)
OUTPUT="manifesto_$USER.txt"

# Write to file
echo "----------------------------------------" > $OUTPUT
echo " Open Source Manifesto" >> $OUTPUT
echo "----------------------------------------" >> $OUTPUT
echo "Date: $DATE" >> $OUTPUT
echo "User: $USER" >> $OUTPUT
echo "" >> $OUTPUT
echo "I use $TOOL as my primary tool." >> $OUTPUT
echo "For me, freedom means: $FREEDOM." >> $OUTPUT
echo "In the future, I want to build: $BUILD." >> $OUTPUT
echo "" >> $OUTPUT
echo "I believe in open-source and sharing knowledge." >> $OUTPUT

# Display output
echo ""
echo "Manifesto saved in file: $OUTPUT"
echo ""
cat $OUTPUT
```

This script takes user input and generates a personalized manifesto. It demonstrates input handling, file writing, and string formatting.

OUTPUT :

```
pathik@PATHIK:/mnt/c/Users/pathi/oss-project$ nano script5.sh
pathik@PATHIK:/mnt/c/Users/pathi/oss-project$ chmod +x script5.sh
chmod: changing permissions of 'script5.sh': Operation not permitted
pathik@PATHIK:/mnt/c/Users/pathi/oss-project$ ./script5.sh
===== Open Source Manifesto Generator =====

1. Which tool do you use? Git
2. What does freedom mean to you? Ability to learn and modify code
3. What do you want to build? Useful software for people

Manifesto saved in file: manifesto_pathik.txt

----------------------------------------
 Open Source Manifesto
----------------------------------------
Date: 28 March 2026
User: pathik

I use Git as my primary tool.
For me, freedom means: Ability to learn and modify code.
In the future, I want to build: Useful software for people.

I believe in open-source and sharing knowledge.
pathik@PATHIK:/mnt/c/Users/pathi/oss-project$ |
```

Conclusion :

After completing this project, I feel that I have gained a deeper understanding of open-source software and how it works in real-world scenarios. Studying Git helped me understand not just version control, but also the philosophy behind open-source development.

I also learned how Linux systems manage software and how shell scripting can be used to automate tasks. Writing and executing scripts gave me practical experience, which made the concepts much clearer.

Overall, this project helped me improve both my theoretical knowledge and practical skills. I believe that tools like Git will be very useful in my future career, and I will continue using and learning more about open-source technologies.

References :

- Official Git Documentation
- Linux Manual Pages
- Open Source Initiative
- Personal Practice and Learning